Explain · for ML Capstone students · 2026-10-21

EXPLAIN · ML CAPSTONE

An architecture worth copying

Virtual Patient Clinic is hello-world-app grown into a real product. It keeps the same few rules across 5 services, 12 DAOs and over 200 tests, and those rules are why 2 people and an AI agent can keep changing it safely. This page starts with the architecture itself, simple enough for any capstone project. Then it shows Virtual Patient Clinic as one instance of it, names the design patterns behind its rules, shows where each one lives in the code, and how to use them in your project.

Read time about 18 minutes | **Code** excerpts from Virtual Patient Clinic (a private repo) | **Version** 0.2.0

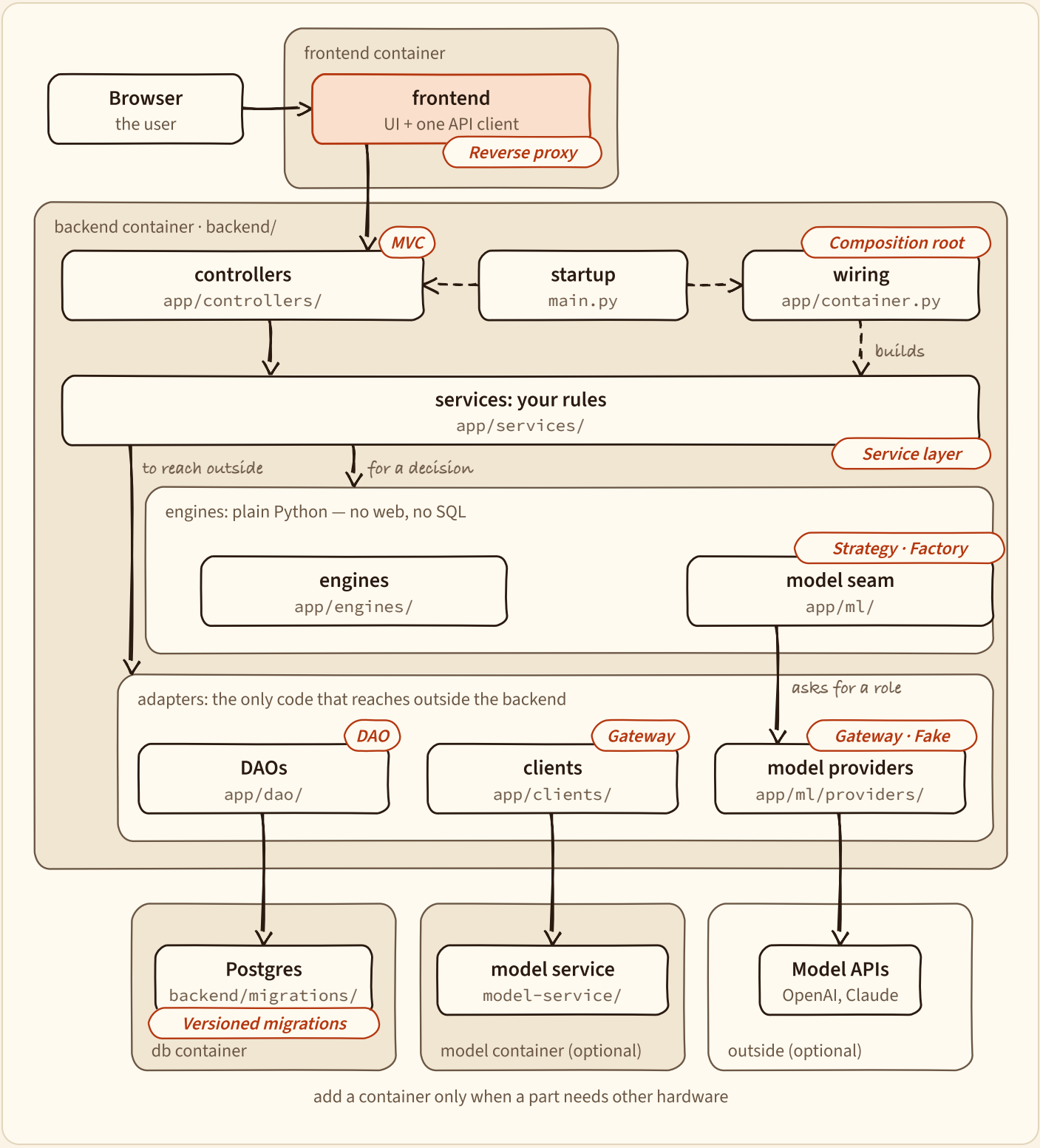
> **What is Virtual Patient Clinic?**

THE ARCHITECTURE

One architecture, many projects

Start with the plan, not the product. This drawing is the architecture any capstone project can use. Virtual Patient Clinic is one instance of it, with a few extra parts.

> **container** One program packed with everything it needs to run, so it runs the same on your laptop and on the server. Each box with a gray outline below is one container.



The reference architecture. Solid arrows are requests: they flow down and never back up. Dashed arrows run once, at startup: `main.py` registers the controllers and calls `container.py`, which builds the services and everything below them. Only the frontend is public, only the backend talks to the database, and only `app/ml/` knows which model answers. The parts marked optional are there when your project needs them.

Read it top to bottom. Each part has one job and one home in the code:

- **frontend** (`frontend/`): the web page. It sends every request to the backend through one API client file and holds no secrets.
- **controllers** (`app/controllers/`): read the request, call one service method, send the answer back. No rules here.
- **services** (`app/services/`): the rules of your app. Everything below it is a helper that the services call.
- **engines** (`app/engines/`): your core logic as plain code. It is easy to test because it never touches the web or the database.
- **model seam** (`app/ml/`): one small interface in front of your ML model. `router.py` reads the config and picks a provider for the role you asked for; it makes no network call itself, which is why it counts as plain code. The call happens one level down, in `app/ml/providers/` — one file per model, and always a fake one for tests.
- **adapters** (`app/dao/` , `app/clients/` , `app/ml/providers/`): the only code that reaches outside the backend — the database, another container, or a hosted model API. Everything above this band is plain Python, which is what makes it easy to test.
- **startup** (`main.py`): registers the controllers' routes, turns rule errors into HTTP codes, and calls `container.py` .
- **wiring** (`app/container.py`): builds all of the above and connects them, in one place.

What a service looks like

The drawing says engines and adapters are peers that never call each other. A service is where that shows up: it holds the order of the steps, and every other line reaches sideways to one helper or the other.

app/services/document_service.py (pseudocode)

```
async def submit(self, user, document_id, text):
    doc = await self.documents.get(document_id)        # adapter
    if doc is None:
        raise NotFound("no such document")             # a rule, not a 404
    if doc["owner_id"] != user["id"]:
        raise Forbidden("not yours")

    cleaned = normalise(text)                          # engine
    if not cleaned:
        raise Invalid("write something first")

    summary = await self.summariser.run(cleaned)       # engine → seam
    await self.search.index(document_id, summary)      # adapter
    return await self.documents.save(doc_id, summary)  # adapter
```

Read the comments down the right-hand side: adapter, engine, engine, adapter, adapter. The service alternates between the two bands because it is the only thing that knows they both exist. `normalise()` cannot read the database and `self.documents` cannot decide anything, so neither can be tempted into the other's job. The one line that goes deeper rather than sideways is `self.summariser.run()`: an engine that asks the seam for a model, which is the single arrow between the bands in the drawing.

Where every file goes

The folders follow the drawing: one folder per container, and inside the backend, one folder per layer. When each kind of code has one home, a change touches one folder, a reviewer knows where to look, and an AI agent puts new code in the right place.

the repo

```
your-project/
  CLAUDE.md                          rules for people
  AGENTS.md                          and for agents
  README.md
  docker-compose.yaml                every container
  docker-compose.override.yml        local changes
  .env.example                       every setting
  smoke-test.sh                      end-to-end check
  .github/workflows/                 tests before deploy
  docs/
    architecture.md                  your map
    reference-architecture.md        agent rules
    STATUS.md                        what works
  frontend/                          frontend container
    src/api/client.ts                the one API client
    src/pages/                       one per screen
    src/components/                  shared parts
  model-service/                     optional, GPU
    engines/                         real.py, fake.py
  backend/                           see the right side
  terraform/                         deploy wiring
```

backend/: one folder per layer

```
backend/
  main.py                            startup only
  app/
    config.py                        env vars, once
    container.py                     builds every part
    db.py                            connection pool
    controllers/                     routes, thin
      deps.py                        container, user
    services/                        the rules
      errors.py                      NotFound, …
    dao/                             the only SQL
    engines/                         core logic
    ml/                              model interface
      providers/                     real + fake
    clients/                         other containers
    models/                          API shapes
  config/models.yaml                 model per role
  migrations/                        001_….sql
  tests/                             api, engine, flow
  Dockerfile
```

One folder per container. Inside the backend, one folder per layer.
A file goes in the folder of the layer it belongs to, never in two.
Change one thing, edit one folder: that is what keeps it easy to maintain.

The reference layout. Bold names are folders. Your project adds files inside these folders, not new folders beside them.

Give it to your coding agent. `reference-architecture.md` writes all of this down as rules for Claude Code or Codex: the layout, the layers, the wiring, the tests, and the order to add a feature in. Put it in your repo as `docs/reference-architecture.md` and point to it from `CLAUDE.md` or `AGENTS.md`. Its last task has the agent draw your project's own map, in this shape, with your real code paths.

Virtual Patient Clinic as an instance

Virtual Patient Clinic fills in every part, sometimes with its own folder names, then adds a few things for its own needs. The added boxes are amber in the next drawing.

REFERENCE PART	IN VIRTUAL PATIENT CLINIC	ADDED FOR THIS PROJECT
frontend `frontend/`	React, served by nginx; `src/api/client.ts`	Voice code in `src/voice/`: microphone, playback, cutting the patient off
controllers `app/controllers/`	Same folder; HTTP routes plus a voice WebSocket	none
services `app/services/`	Same folder: cases, encounters, scoring, voice, faculty review	none
engines `app/engines/`	One folder per engine: `app/patient/`, `app/evaluation/`, `app/authoring/`	none
model seam `app/ml/router.py`	`app/llm/router.py` with `config/models.yaml`: picks a provider for the role, no network call	none
DAOs `app/dao/`	Same folder	none
model providers `app/ml/providers/`	`app/llm/providers/`: `openai_compat.py`, `anthropic_provider.py`, `fake.py` — where the model call actually happens	none
clients `app/clients/`	`app/speech/client.py`, `app/avatars/simli.py`	`app/realtime/hub.py` for live watching; `app/storage/` for recordings, turned off
db	Postgres 16, `backend/migrations/`	An event log: every turn is stored, and scores are worked out from it
model service `model-service/`	`speech/`: Whisper and text to speech on a GPU	A second GPU container, `localsimli/`, for the video face. Its video goes straight to the browser, an approved exception to "only the frontend is public"

REFERENCE PART	IN VIRTUAL PATIENT CLINIC	ADDED FOR THIS PROJECT
Model APIs	OpenAI and Claude	none

Your project is another instance. Keep the parts and the rules. Change what is inside each box: your engines, your model, your tables.

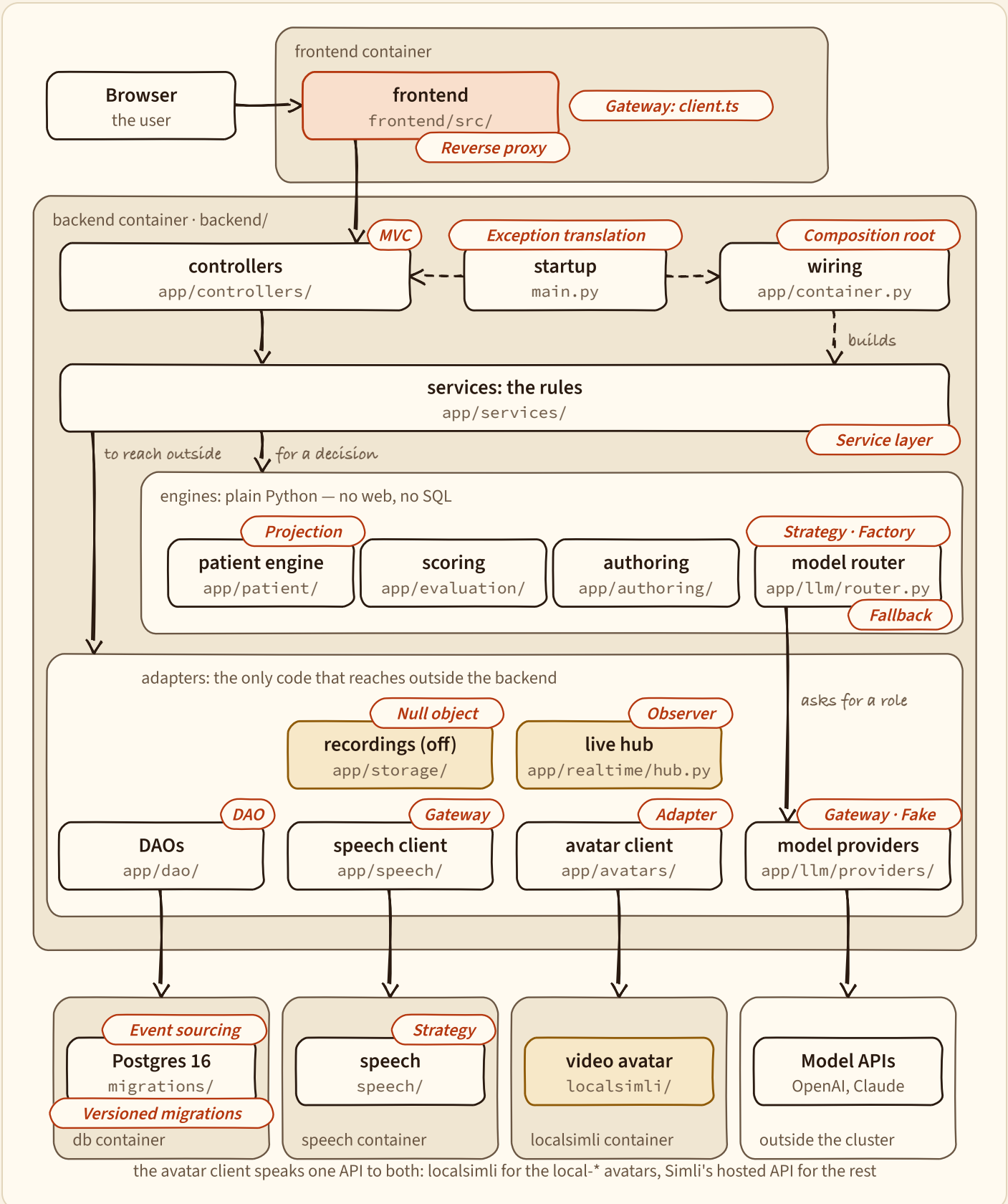
THE MAP

The patterns, by name

Here is the same architecture with Virtual Patient Clinic filled in. Most of what makes this code easy to change is a known design pattern. Each one has a name, so you can look it up, talk about it, and use it on purpose.

> **design pattern** A named, proven answer to a problem that keeps coming up in code, such as "how do I swap one model for another without changing the callers?"

Some names come from the 1994 book *Design Patterns* by the "Gang of Four" (marked GoF below). Some come from Martin Fowler's *Patterns of Enterprise Application Architecture* ("enterprise"). The rest are common names for architecture, testing and operations.



Virtual Patient Clinic on the reference architecture. Amber boxes are parts the reference doesn't have. Each outlined box is a container; the backend's layers run top to bottom, and dashed arrows are the startup wiring from section 3. `services/` sits above two bands: engines that are plain Python, and below them the adapters. Every arrow that leaves the backend starts in that lower band, directly above the container it reaches, and arrows only point down. The two

bands are peers under the services, not a stack: the services call an engine for a decision and an adapter to reach outside, and the only call between the bands is the router asking for a provider. An engine never calls an adapter — the patient engine does not touch the speech client or the avatar client, because a service does that. Its one way out is to ask the model router for a role, and the router hands back a provider. Each badge is a pattern, pinned to the part of the code where it lives. Sections 1 to 7 take them one at a time.

One spoken turn, through both bands

Here is the same shape in the real code. A student speaks; the patient answers. Follow the comments and you can watch the service cross between the bands.

backend/app/services/voice_service.py (trimmed)

```
async def handle_audio_turn(self, encounter_id, user, audio, mime, send):
    doc = await self.encounters.active_case(...)               # a service
    heard = await self.speech.transcribe(audio, ...)           # adapter
    if not heard.text:
        return await send({"type": "no_speech"})
    delivery = measure(heard.text, heard.speech_s, ...)        # engine
    await self._run_turn(..., doc, send, delivery=delivery)

# inside _run_turn, once per finished sentence:
    audio, mime = await self.speech.synthesize(sentence, ...)  # adapter
```

That service owns the voice path and nothing else. The turn itself belongs to another service, which is allowed: a service may call a service for a step it already owns.

backend/app/services/encounter_service.py (trimmed)

```
async def student_turn(self, encounter_id, user, text, source, delivery):
    doc = await self.active_case(encounter_id, user)            # adapter
    await self._append(encounter_id, "student_utterance", ...)  # adapter
    events = await self.encounters.events(encounter_id)         # adapter

    async for delta in self.engine.reply(doc, events, ...):     # engine
        shown = cues.feed(delta)                                # engine
        self.hub.publish(observe_topic(encounter_id), ...)      # adapter
        yield TurnEvent("patient_delta", {"text": shown})

    await self._append(encounter_id, "patient_utterance", ...)  # adapter
```

Adapter, engine, adapter, engine, interleaved by the service, and not one call between them. The patient engine never touches the speech client, even though they serve the same turn: `voice_service` transcribes the audio and passes the words in. The only line that leaves the engine band is `self.engine.reply()` reaching the model router, which hands back a provider.

All 22 patterns, and where each is explained

PATTERN	FAMILY	EXPLAINED IN
Layered architecture	architecture	Section 1
Reverse proxy	infrastructure	Section 1
Model-View-Controller (MVC)	architecture	Section 2
Service layer	enterprise	Section 2
Data access object (DAO)	enterprise	Section 2
Exception translation	enterprise	Section 2
Dependency injection, composition root	architecture	Section 3
Test doubles: mock and fake	testing	Section 3, 4
Null object	GoF-style	Section 3
Strategy	GoF	Section 4
Factory	GoF	Section 4
Adapter	GoF	Section 4
Fallback (failover)	reliability	Section 4
Event sourcing	architecture	Section 5
Projection	event sourcing	Section 5
Allow list (deny by default)	security	Section 5
Observer (publish/subscribe)	GoF	Section 5

PATTERN	FAMILY	EXPLAINED IN
Producer/consumer	concurrency	Section 5
Gateway	enterprise	Section 6
Single source of truth	design rule	Section 6
Versioned migrations	data	Section 7
Health check (liveness and readiness)	operations	Section 7

Patterns are tools, not goals. Each one here fixed a problem the app really had. Reach for one when you have its problem, not to fill in a checklist.

1 · THE SHAPE

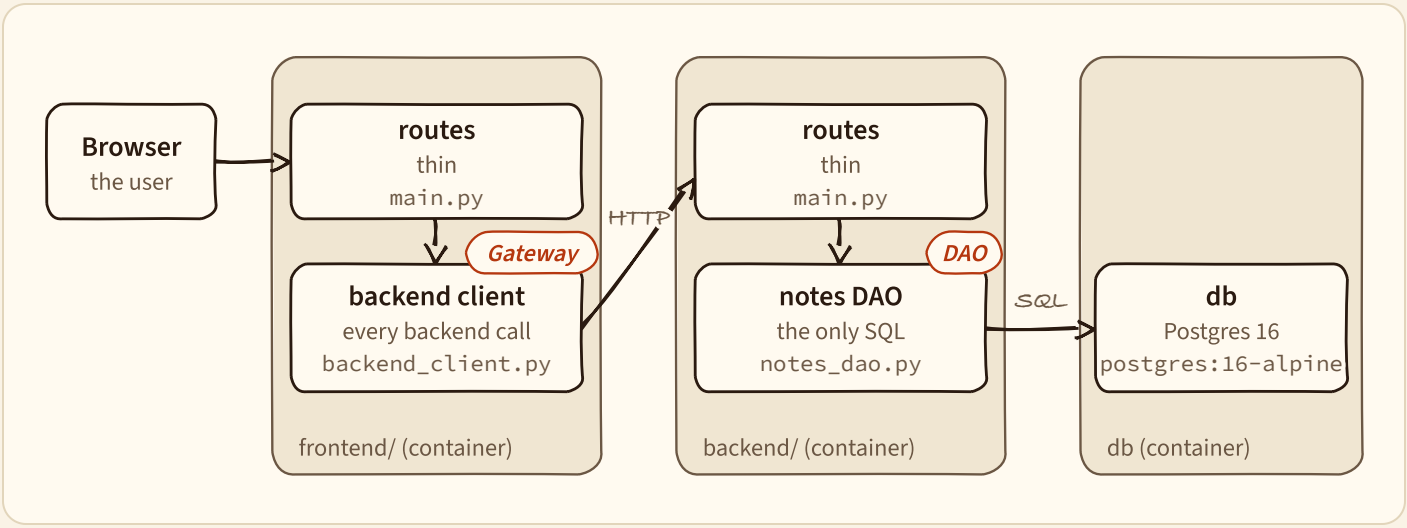
It is hello-world-app with more services

You already know this layout. Virtual Patient Clinic adds services to it, but it does not bend the rules.

frontend The part of an app that runs in the browser. The **backend** is the part on the server that does the work and keeps the data.

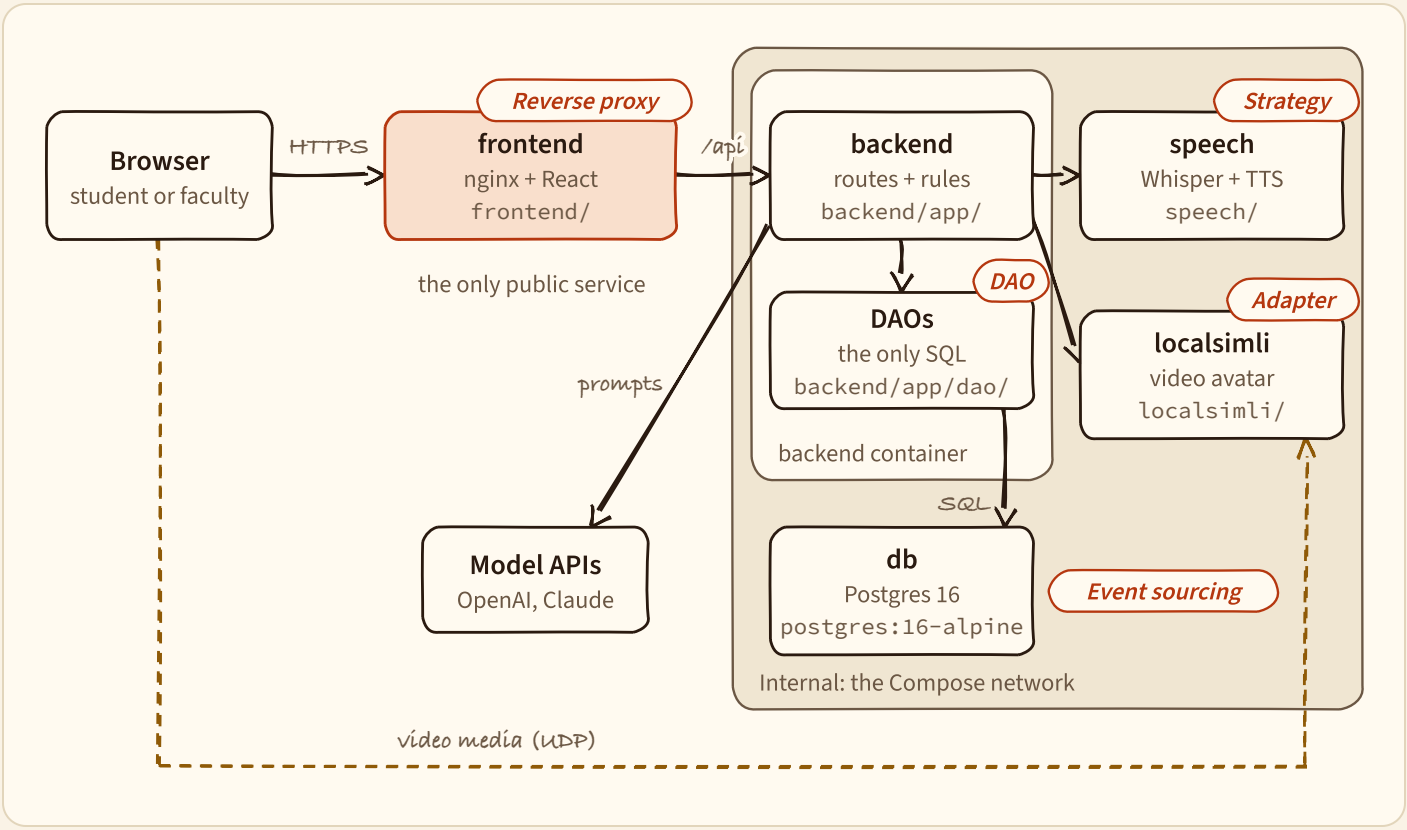
service One program in its own container, with its own folder, Dockerfile and port. Compose starts them all and lets them find each other by name.

In hello-world-app, a request goes one way: frontend, then backend, then database. Only the frontend is public, and only the backend has the database password. Virtual Patient Clinic keeps exactly that, then hangs more services off the backend.



hello-world-app. Each container has thin routes in `main.py` and one boundary file: `frontend/backend_client.py` is the only code that calls the backend, and `backend/notes_dao.py` is the only code that talks to the database. The db container runs the stock `postgres:16-alpine` image.

Virtual Patient Clinic has the same shape. Its backend grew past two files, so its code moved into `backend/app/` with one folder per layer, and `notes_dao.py` became the `app/dao/` folder.



Virtual Patient Clinic. Everything goes through the backend. Inside the backend, the DAOs are the only code that talks to the database, the job `notes_dao.py` does in hello-world-app. The line in code font is where each part lives: a folder in the repo, or the Docker image for the database. The browser can reach only the frontend. The one exception is the dashed line: live video runs straight from the browser to the video service. We approved that exception on purpose, and the docs say why.

hello-world-app

3 services: `frontend`, `backend`, `db`. One boundary file in each service: `backend_client.py` and `notes_dao.py`.

Virtual Patient Clinic

5 services: the same 3, plus `speech` and `localsimli` on the GPU. Still one owner for the data, and still one public service.

SERVICE	PUBLIC?	WHAT IT OWNS
`frontend/`	yes	The React app, served by nginx. nginx also passes `/api` and the voice WebSocket to the backend. It holds no secrets.
`backend/`	no	Every rule, all the SQL, and every call to a model API.
`db`	no	Postgres 16. The backend owns the schema through numbered SQL files.
`speech/`	no	Speech to text (Whisper) and text to speech on the GPU. Stores nothing.
`localsimli/`	no*	The video face, on the GPU. *Its video stream uses a small range of UDP ports.

Patterns in this section

PATTERN	WHERE	HOW IT FITS
Layered architecture	The 5 services; inside the backend, controllers, services and DAOs	Each layer talks only to the layer below it. You can change or test one layer without touching the others.
Reverse proxy	nginx in `frontend/` passes `/api` and the voice WebSocket to `backend:8001`	The browser sees one address. The backend never gets a public URL.
Strategy, Adapter	`speech/` (swappable engines) and `localsimli/` (our server speaking Simli's API)	Explained in section 4. The badges show where they sit in the system.

Why the rule pays off. The frontend faces the internet, so it is the part most likely to be attacked. It has no database driver, no password and no API keys, so a bug there cannot leak what it cannot reach.

2 · INSIDE THE BACKEND

Each layer in the backend has one job

The backend is split into layers. A request passes down through them, and each layer is allowed to do one kind of work.

PICTURE IT

A restaurant. The waiter takes your order and brings the plate, but never cooks. The chef decides how each dish is made. Only the storeroom clerk goes into the storeroom. If the clerk moves the flour, nobody else has to learn where it went.

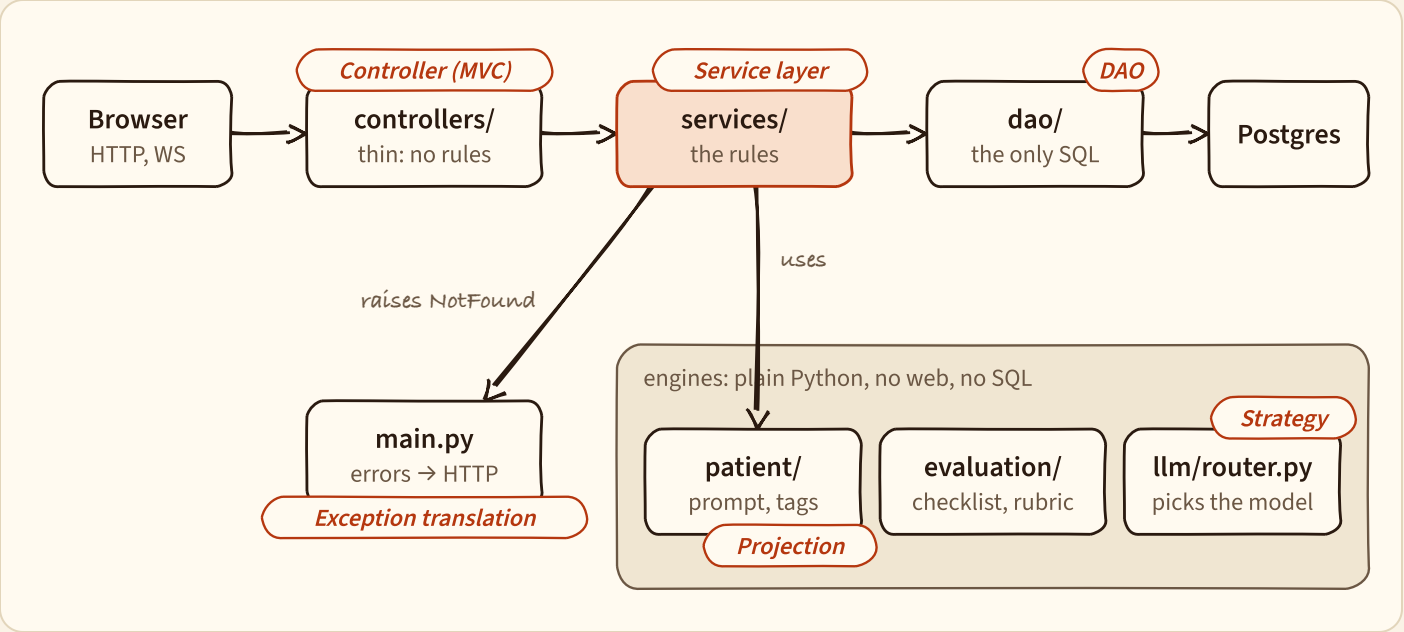
THE REAL THING

The **controller** is the waiter: it reads the request, calls one service method, and returns the answer. The **service** is the chef: it holds the rules, such as who may start which encounter. The **DAO** is the clerk: it holds the only SQL in the app.

WHERE THE PICTURE BREAKS

In a real kitchen, a busy chef grabs the flour. In code, that shortcut is the bug. A rule in a controller or SQL in a service works today, then gets copied, and soon the same rule lives in 3 places that disagree.

DAO Data access object: a class whose methods run SQL and return plain rows. One DAO per kind of data, such as cases or encounters.



Arrows point one way: down. Services also use engines, which are plain Python with no web code and no SQL. When a rule fails, the service raises an error and `main.py` turns it into an HTTP code.

Here is one small feature, feedback, at all 3 layers. The controller is 2 lines per route:

backend/app/controllers/feedback.py

```
@router.post("/feedback", status_code=201)
async def send(body: FeedbackIn, user=Depends(current_user), c: Container =
Depends(container)):
    return await c.feedback.send(user, body.kind, body.text, body.route,
body.encounter_id)
```

The service holds the rules. It knows nothing about HTTP. It raises `Invalid`, not a 422:

backend/app/services/feedback_service.py

```
async def send(self, user, kind, text, route, encounter_id):
    if kind not in KINDS:
        raise Invalid(f"kind must be one of {', '.join(KINDS)}")
    text = text.strip()
    if not text:
        raise Invalid("write a sentence or two before sending")
    ...
    return await self.feedback.create(user["id"], kind, text, route,
encounter_id)
```

The DAO holds the SQL, and nothing else:

backend/app/dao/feedback_dao.py

```
async def create(self, user_id, kind, text, route, encounter_id) -> dict:
    async with self.db.connection() as conn:
        cur = await conn.execute(
            "INSERT INTO feedback (user_id, kind, text, route, encounter_id)
"
            "VALUES (%s, %s, %s, %s, %s) RETURNING id, kind, status,
created_at",
            (user_id, kind, text, route, encounter_id),
        )
        return await cur.fetchone()
```

⚠ **Services never import FastAPI.** That one rule keeps them testable without a web server. It also lets the same service answer the web page, the WebSocket and a script, and each one gets the same error.

Patterns in this section

PATTERN	WHERE	HOW IT FITS
Model-View-Controller (MVC)	Model: `app/models/` (pydantic shapes). View: the React pages. Controller: `app/controllers/`	Splits what the data is, how it looks and how requests are handled, so each can change alone.
Service layer	`app/services/`	One place for the rules, used by every way in: HTTP routes, the WebSocket and scripts.
Data access object (DAO)	`app/dao/`, one per kind of data	Hides SQL behind plain methods. Services ask for data; they never write a query.
Exception translation	Services raise `NotFound`, `Forbidden`… (`services/errors.py`); one handler in `main.py` maps them to 404, 403…	Services stay free of HTTP. Every route returns the same error shape.

BIGGER IDEA

This is MVC with a DAO layer, the same shape as Rails, Django and Spring apps. Learn it once and you can find your way around most backends you will meet at work.

3 · WIRING AND TESTS

One file builds everything, so tests can swap the parts

No class creates its own helpers. One function, `build_container()`, creates every DAO and service once and hands each one what it needs.

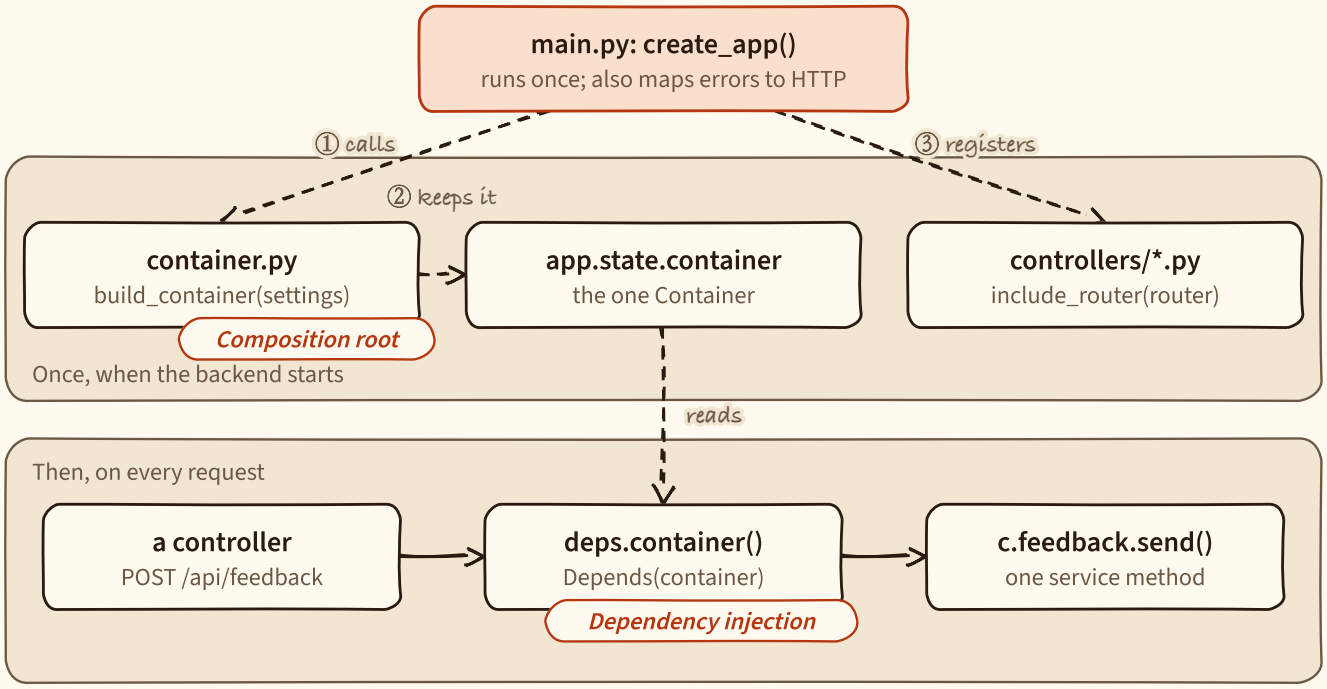
> **composition root** The one place in an app that creates the objects and connects them. Everything else gets its helpers passed in.

backend/app/container.py (trimmed)

```
def build_container(settings: Settings) -> Container:
    db = Database(settings.database_url)
    router = ModelRouter.from_config(settings.model_config_path, ...)
    case_service = CaseService(CasesDAO(db), settings.seed_dir)
    evaluation = EvaluationService(EvaluationsDAO(db), ..., router)
    return Container(
        cases=case_service,
        feedback=FeedbackService(FeedbackDAO(db), encounters),
        ...
    )
```

How `main.py` puts it together

`main.py` holds no routes and no rules. Its one function, `create_app()`, runs once when the backend starts and does three things: it calls `build_container()`, keeps the result on `app.state`, and registers each controller file's routes. After that, every request finds its service through that one container.



Dashed lines run once, at startup. Solid lines run on every request. A controller never builds a service; it asks the container for one.

backend/main.py (trimmed)

```
def create_app(settings=None, container=None) -> FastAPI:
    app = FastAPI(lifespan=lifespan)       # lifespan: open the db, run
migrations, seed
    app.state.container = container or build_container(settings)   # ① and
②

    @app.exception_handler(ServiceError)   # a rule failed: NotFound becomes
404, ...
    async def service_error(_, exc):
        return JSONResponse(status_code=exc.status_code, content={"detail":
exc.message})

    for module in (auth, users, cases, encounters, feedback, ...):
        app.include_router(module.router)  # ③ each controller file's routes
    return app
```

backend/app/controllers/feedback.py and deps.py (trimmed)

```
router = APIRouter(prefix="/api")

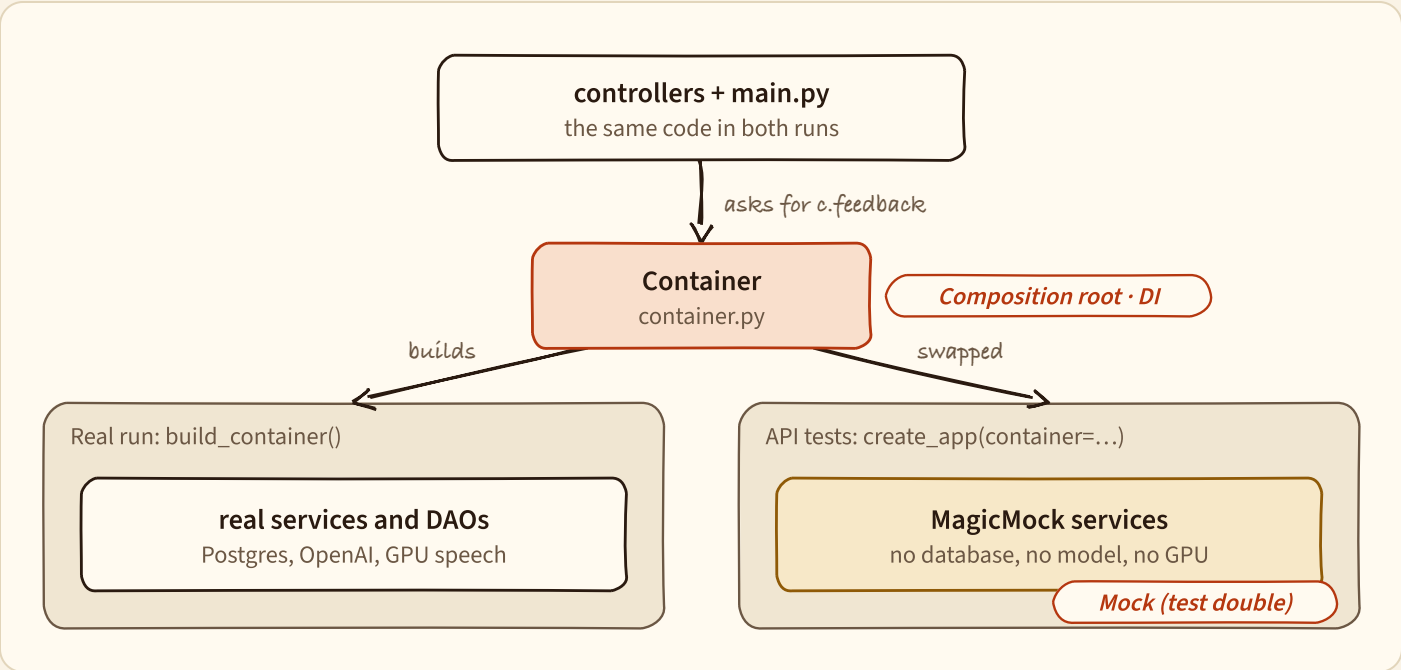
@router.post("/feedback", status_code=201)
async def send(body: FeedbackIn, user=Depends(current_user),
               c: Container = Depends(container)):     # FastAPI calls
deps.container()
    return await c.feedback.send(user, body.kind, body.text, ...)

def container(request: Request) -> Container:            # in deps.py
    return request.app.state.container
```

The `container=None` argument is the key to testing. A real run leaves it empty, so `create_app` builds the real parts. A test passes its own.

Because every part comes from this one place, a test can build the app with different parts. The API tests hand `create_app` a container full of mocks. They check the controllers and the error codes, with no database, no model and no GPU.

> **mock** A stand-in object for tests. You tell it what to return, then check how it was called.



The controllers are the same code in both runs. Only the container changes.

Example: a test that a rule becomes the right HTTP code

1. Build the app with a mock container: `create_app(settings, container=c)`.
2. Make the mock service fail: `c.encounters.start = AsyncMock(side_effect=Conflict("no attempts remaining"))`.
3. Call `POST /api/encounters` and check for a 409 with that message.

The test runs in milliseconds, and the backend's 200+ tests take seconds, so people actually run them.

Patterns in this section

PATTERN	WHERE	HOW IT FITS
Dependency injection	Every service takes its DAOs and helpers as constructor arguments; controllers get the container through `Depends(container)`	A class never builds its own helpers, so a test or a config change can hand it different ones.
Composition root	`app/container.py`: `build_container()`, called once by `create_app()` in `main.py`	The one place where the real parts get chosen and connected.
Null object	`NullRecordingStore` in `app/storage/recordings.py`	With audio saving off, the container passes a store whose `save` does nothing. No `if retain_audio:` checks spread through the code.
Test double: mock	`MagicMock` and `AsyncMock` services in `tests/test_api.py`	Tests the controllers alone: fast, and no database.

KIND OF TEST	WHAT IT SWAPS	WHAT IT PROVES
API tests	Every service is a mock	Routes, sign-in checks and error codes
Engine tests	The model is the fake provider	Prompts, tag parsing and scoring logic
Integration test	The model and speech are fakes; Postgres is real	A full encounter, from start to report, against a real database

4 · SEAMS

Put anything you might swap behind one small interface

The app has used Claude, OpenAI, the class cluster and a fake model. Switching takes no code change, because no code outside one folder knows which model it is talking to.

PICTURE IT

A wall socket. Your lamp does not care whether the power comes from a dam or a solar farm. It cares only that the socket has the usual shape.

THE REAL THING

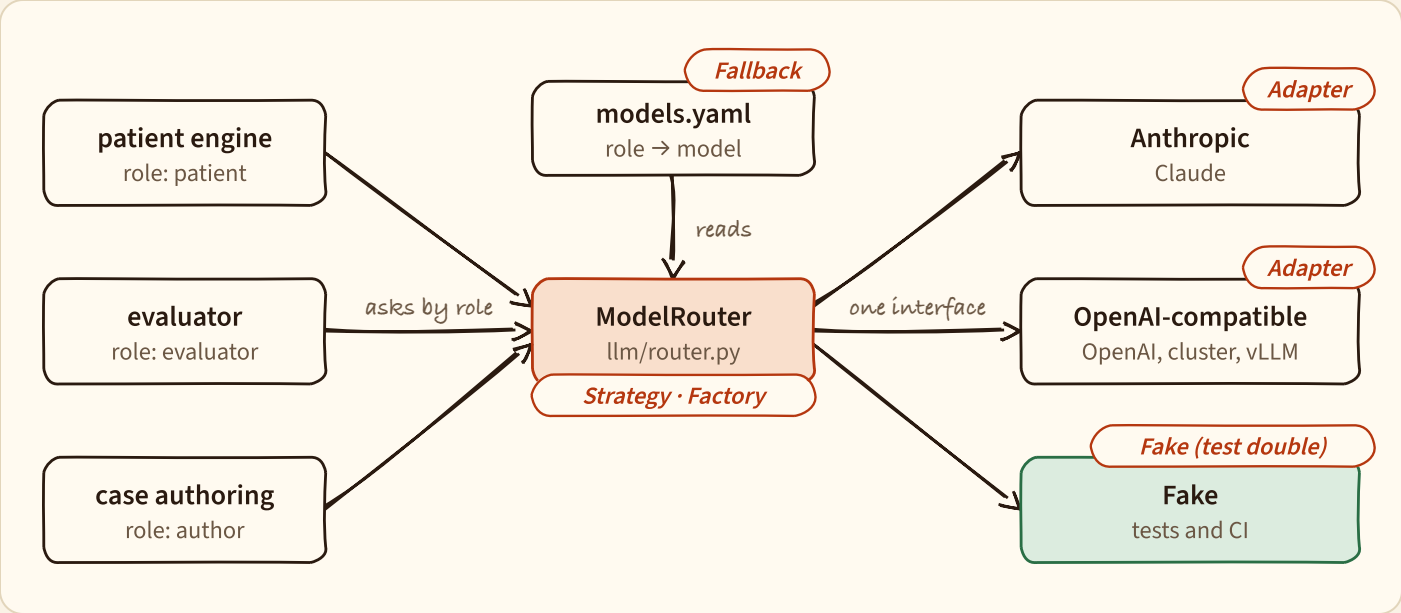
Code asks for a **role**, such as `patient` or `evaluator`. The **model router** looks up the role in `models.yaml` and calls a **provider**. Every provider has the same 3 methods: `available`, `complete` and `stream`.

WHERE THE PICTURE BREAKS

Power is power, but models are not all the same. The fake model proves the plumbing works, not that the patient sounds real. You still have to test the real model.

fake A small, simple version of a part that works but gives canned answers, such as a model that always says the same thing. Tests and laptops use it in place of the real one.

interface A short list of methods that several classes all provide. Code that uses the interface works with any of those classes.



3 parts of the app ask by role. The config file decides who answers. The fake provider lets CI run every test with no API key.

backend/config/models.yaml (trimmed)

```
roles:
  patient:   {provider: anthropic, model: claude-sonnet-5-5, fallback:
{provider: cluster, model: classroom-chat}}
  evaluator: {provider: anthropic, model: claude-opus-5-5,   fallback:
{provider: cluster, model: classroom-chat}}
```

The router does more than pick a model. It falls back when a provider is down, it times every call, and it logs token use per encounter. Because every call goes through it, you get all of that in one place and for free.

The same idea shows up all over the app:

- **Speech:** real Whisper and TTS on the GPU, or fake engines for CI and laptops (`STT_ENGINE=fake`).
- **Video face:** several avatar types behind one `AvatarRenderer` interface in the browser.
- **Sign-in:** local passwords today, with an `OIDCProvider` stub waiting for BYU single sign-on.

Patterns in this section

PATTERN	WHERE	HOW IT FITS
Strategy	`LLMProvider` in `app/llm/providers/base.py`; `STTEngine` and `TTSEngine` in `speech/engines/base.py`; `AvatarRenderer` in `frontend/src/avatar/renderer.ts`	Several classes do the same job in different ways behind one interface. The caller picks one and never knows which.
Factory	`build_provider()` and `ModelRouter.from_config()` in `app/llm/router.py`	Turns a line of config into the right provider object, so callers never write `if vendor == ...`.
Adapter	Each provider class wraps a vendor SDK; `localsimli/` serves Simli's API on top of MuseTalk	Makes something with the wrong shape fit the interface we need. Because localsimli speaks Simli's API, the app's Simli client works with it unchanged.
Fallback (failover)	`fallback:` in `models.yaml`, used by the router	If the first provider can't answer, the next one does. The caller never sees it.
Test double: fake	`FakeProvider`, and `STT_ENGINE=fake` / `TTS_ENGINE=fake`	A real, simple stand-in, so CI runs the whole app with no key and no GPU.

Never call a vendor's SDK outside its folder. One stray `openai.chat(...)` in a service skips the fallback, the timing and the tests, and it ties you to that vendor.

5 · DATA

Store what happened, then work out the rest

An encounter is not saved as a big record that gets edited. It is a list of events that only grows. Everything else is worked out from that list.

PICTURE IT

A bank statement. The bank does not just store your balance. It stores every deposit and every charge, and the balance is the sum. If a number looks wrong, you can trace it back to the line that caused it.

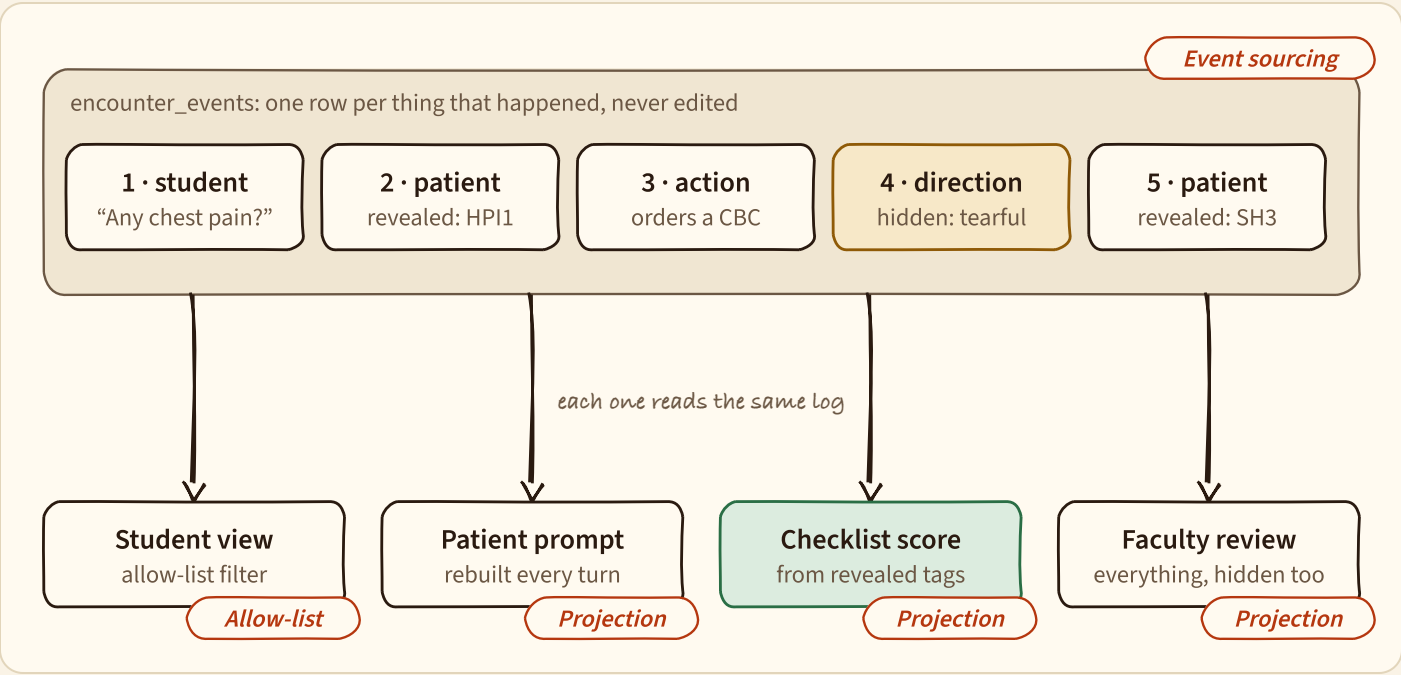
THE REAL THING

The `encounter_events` table gets one row for each thing that happened: what the student said, what the patient said and which facts that reply revealed, each test ordered, and each change a watching instructor made. Rows are added and never changed.

WHERE THE PICTURE BREAKS

Working everything out from the log each time costs time. So the app also stores results that are slow to make, like the AI's evaluation, and it treats them as reports built from the log.

append-only New rows are added; old rows are never updated or deleted. The table is a history, not a current state.



One log, four readers. Event 4 is hidden: the student never sees it, but the patient's next reply follows it.

One list does 4 jobs:

- **The transcript** the student and faculty read.
- **The patient's memory:** the prompt is rebuilt from the log on every turn, so the patient engine stores nothing.

- **The score:** the checklist counts the facts the patient revealed, so every point traces back to a turn number.
- **Live steering:** an instructor's change is just another hidden event, so it can be replayed exactly.

Hidden data needs a guard. Before events go to a student, one function keeps only the kinds on a short allow list and strips the hidden fields:

backend/app/services/encounter_service.py

```
def _student_safe(events):
    """Drop hidden event kinds and strip disclosure bookkeeping before
events
    go to the browser. Allow-list, so a new kind is hidden until opted
in."""
    for ev in events:
        if ev["kind"] not in STUDENT_KINDS:
            continue
        ...
```

Allow lists fail safe. A block list ("hide these kinds") leaks the first new kind someone adds and forgets. An allow list ("show only these") hides it until someone decides it is safe.

Patterns in this section

PATTERN	WHERE	HOW IT FITS
Event sourcing	The `encounter_events` table	Store every change as an event. The current state is whatever the events add up to.
Projection	`_student_safe()`, the checklist scorer, the prompt builder, and `effective_patient()` in `patient/steering.py`	Each reader folds the same events into the view it needs. A new view needs no new storage.
Allow list (deny by default)	`STUDENT_KINDS` in `patient/steering.py`	A new event kind stays hidden from students until someone adds it to the list.

PATTERN	WHERE	HOW IT FITS
Observer (publish/subscribe)	`app/realtime/hub.py`	Every new event is published to anyone listening, without the student's socket knowing who. The instructor's live-watch page will be the first listener.
Producer/consumer	`services/voice_service.py`	One task streams the model's text into a queue a sentence at a time. Another turns each sentence into speech. The student hears the first sentence while the model is still writing the rest.

BIGGER IDEA

The app uses append-only in 3 places: the event log, faculty score changes (stored as a history, not over the old score), and database migrations (a change is always a new numbered file, never an edit to an old one). It is the same idea as git: keep the history and work out the present from it.

6 · TEAMWORK

Write down the rules that cross files

Some changes touch more than one file. The repo lists them in a table, so nobody, human or AI agent, has to remember.

> **contract** An agreement between 2 or more files about a shape, such as the JSON a route returns. Change one side and the other must change too.

The table lives in `CLAUDE.md` at the root of the repo. Claude Code reads that file at the start of every session, and people read it too. Here are 3 of its rows:

IF YOU CHANGE…	…ALSO UPDATE
Any API route or response	`frontend/src/api/client.ts` and `types.ts`, plus tests in `test_api.py`

IF YOU CHANGE…	…ALSO UPDATE
The database schema	A **new** file `backend/migrations/NNN_description.sql`. Never edit one that has run.
A runtime setting (env var)	`config.py`, both compose files, `.env.example` and `docs/deployment.md`

The frontend follows the boundary rule too. Every call to the backend goes through `src/api/client.ts`, just as every call in hello-world-app goes through `backend_client.py`. Pages never build URLs or read status codes themselves.

Patterns in this section

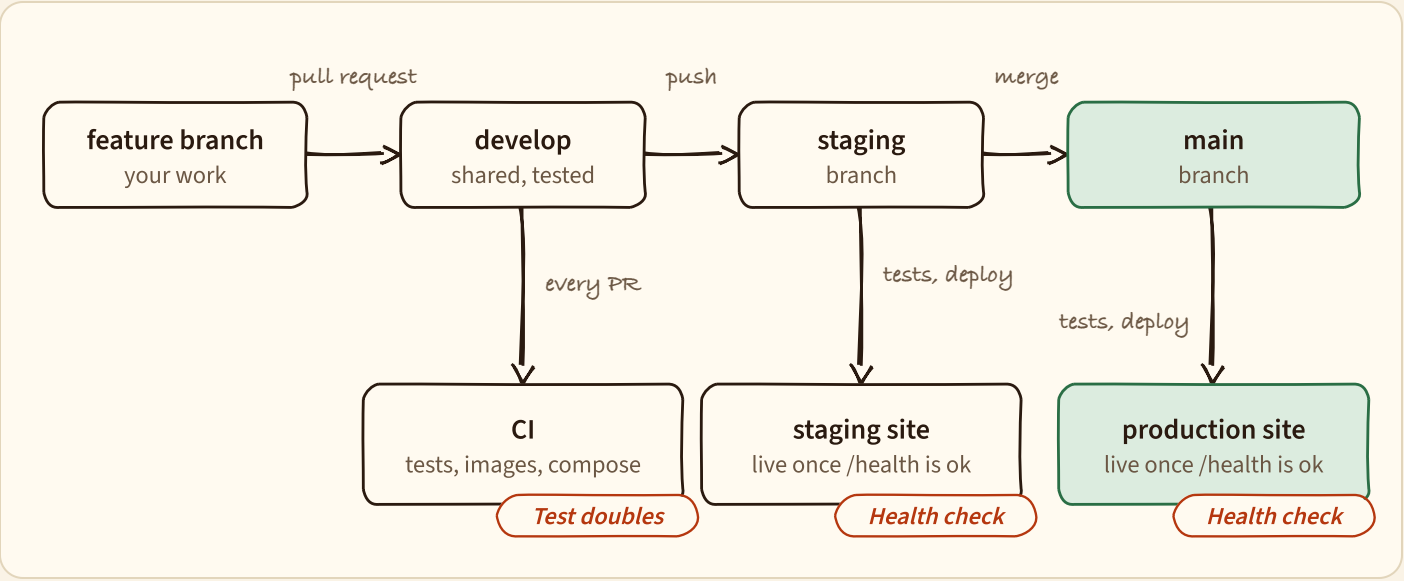
PATTERN	WHERE	HOW IT FITS
Gateway	`frontend/src/api/client.ts`, `app/speech/client.py`, `app/avatars/simli.py`	One class owns all the talk with one outside system. URLs, status codes and retries live there and nowhere else.
Single source of truth	`CaseDocument` in `app/models/case.py` for the case shape; `models.yaml` for models; the contract table for rules that span files	Each fact is defined once. Everything else reads it or copies it on purpose, and the table says which copies to update.

Use the same files in your repo. `CLAUDE.md` holds the rules and the contract table. `docs/STATUS.md` holds what is done, what has never been tried for real, and what comes next. Both help a new teammate, and both make an AI agent far more useful on day one.

7 · SHIPPING

Nothing reaches users without passing the tests

The deploy path is the template's path, with more test jobs. Every service runs its tests in CI on every pull request.



Work moves left to right. Each step runs the same CI. Coolify swaps in the new version only after its health check answers.

- **CI runs everything with fakes:** backend tests (with a real Postgres for the integration test), speech tests, video tests, the frontend type check and build, Docker builds of all 4 images, and a compose check. Any failure stops the deploy.
- **The schema comes along by itself:** at startup the backend runs any numbered SQL file it has not run yet.
- **`/health` versus `/ready`:** `/health` says "this process is up" and checks nothing else, so a slow database cannot roll back a good deploy. `/ready` checks the whole stack and is allowed to fail.
- **One script checks a live site:** `./smoke-test.sh <url>` signs in and walks an encounter on staging or production.

Patterns in this section

PATTERN	WHERE	HOW IT FITS
Health check (liveness and readiness)	`/health` and `/ready` on each service	Liveness says the process is up, so Coolify can switch traffic. Readiness says the whole stack works, so you can tell what is broken.

PATTERN	WHERE	HOW IT FITS
Versioned migrations	`backend/migrations/NNN_*.sql`, run at startup	Each schema change is a new numbered file, so every copy of the database, from your laptop to production, gets the same changes in the same order.
Test doubles in CI	Fake model, fake speech and fake video engines in `ci.yml`	Every test runs on every push with no GPU and no API key.

USE IT

How to bring this into your project

You do not need all of it on day one. Add each piece when your project first needs it.

1

Start from the reference architecture

Give your agent `reference-architecture.md` and ask it to draw your map once the first version works.

- One folder per service, one public service, and only the backend talks to the database.
- Use `expose:`, not `ports:`, in the production compose file.

2

Split the backend when it grows past one file

- Make `controllers/`, `services/` and `dao/` folders, and one error class per HTTP code in `services/errors.py`.
- Map those errors to HTTP codes in one handler in `main.py`.

CHECK `grep -rn "SELECT\|INSERT" app/services app/controllers` prints nothing.

3

Add a composition root

- One `container.py` that builds every DAO and service.
- Let `create_app` take a container, so tests can pass in mocks.

4

Put your ML model behind an interface, with a fake

- Whether it is an LLM, a classifier or a vision model, make one small interface and a config file that picks the real one.
- Write a fake that returns canned answers, so CI and your teammates' laptops never need a GPU or a key.

5 Keep a history where history matters

- If users will ask "why did it say that?", store events, not just the final state.
- Every schema change is a new numbered migration.

6 Write the rules down

- A `CLAUDE.md` with your layers, your contract table and your commands.
- A `docs/STATUS.md` that says what works and what does not.

> ⚠ **Don't copy the size.** Virtual Patient Clinic has 5 services because it needs a GPU for speech and video. Most capstone projects need 3, maybe 4. Add a service when a part needs different hardware, a different language, or its own scaling, not just to look organized.

RECAP

What to remember

1. Requests go one way, and the public service holds no secrets.
2. Controllers stay thin, services hold the rules, and DAOs hold the only SQL.
3. One file builds everything, so tests can swap any part for a mock or a fake.
4. Put anything you might swap, above all your ML model, behind one small interface.
5. Store what happened, write down the rules that cross files, and ship only through CI.
6. Learn the pattern names. They let you say a lot in a few words: "the providers are Strategies, built by a Factory, behind a Fallback."

WORDS

Glossary

append-only

Rows are added, never changed or deleted. The table is a history.

composition root

The one place that creates the app's objects and connects them (`container.py`).

contract

An agreement between files about a shape. Change one side, change the other.

controller

The layer that reads a request, calls one service method and returns the result.

DAO

Data access object: the only code that runs SQL.

design pattern

A named, proven answer to a problem that keeps coming up in code.

fake

A simple working version of a part, for tests (the fake model, the fake speech engines).

interface

A short list of methods that several classes provide, so callers can use any of them.

mock

A test stand-in that returns what you tell it and records how it was called.

service

Two meanings: a container in Compose, or a class in `services/` that holds the rules. The page says which.

SOURCES

Where to read more

> The reference architecture as rules for your coding agent, ready to copy into your repo.
> `reference-architecture.md`
>
> The platform's deployment rules (one public service, health checks, volumes).
> `Migrating Your Project`
>
> The Gang of Four patterns: Strategy, Factory, Adapter, Observer.
> `Refactoring.Guru pattern catalog`
>
> Service layer, DAO (Table Data Gateway), Gateway.
> `Fowler, Catalog of Patterns of Enterprise Application Architecture`
>
> Event sourcing.
> `Fowler, Event Sourcing`

> The template this all grew from.
> Template App README

YOUR TURN

What's still unclear?